August-November 2024

CS616: Statistical Pattern Recognition

CS612: Statistical Pattern Recognition Laboratory

Programming Assignment 3 Report

Submitted by Group 03

Pawan Ram Mali (PhD)
Department of CSE
Indian Institute of Technology
Dharwad, India
cs24dp011@iitdh.ac.in

Abhishek Sharma (MS)
Department of CSE
Indian Institute of Technology
Dharwad, India
cs24ms003@iitdh.ac.in

Sachin Maurya (MS)
Department of CSE
Indian Institute of Technology
Dharwad, India
cs24ms002@iitdh.ac.in

Table of Contents

- Comparative analysis of model performances across different complexities and regularization parameters.
- Analysis of training vs test error trends to understand overfitting and model generalization.

Conclusions

- Summary of findings, effectiveness of regularization, model complexity, and best performing models.

References

List of Figures

1. Introduction

In this assignment, we explore regression techniques on synthetic datasets using methods from statistical pattern recognition. The primary objective is to implement models from scratch, without relying on high-level libraries, to gain a deep understanding of regression techniques and the trade-offs involved in model complexity and regularization.

We focus on two distinct models across different types of datasets:

1. **Polynomial Curve Fitting**: Applied to 1-dimensional synthetic data, this model allows us to explore the impact of polynomial degree on model flexibility and overfitting. By varying the polynomial order, we investigate how model complexity affects training and generalization errors, and we apply regularization to control overfitting for high-complexity models.
2. **Gaussian Basis Linear Regression**: Applied to a 2-dimensional dataset, this model employs Gaussian basis functions with centers derived from clustering. Using the K-means algorithm, we determine cluster centers and use these as centers for Gaussian basis functions. This approach enables us to control model complexity through the number of basis functions and to apply regularization for enhanced generalization.

The report provides an in-depth comparison of models based on their training and testing performance, alongside graphical representations of approximated functions, decision regions, and error metrics. By analyzing these models, we aim to understand the trade-offs between model complexity, regularization, and generalization in regression tasks.

2. Dataset Description

This study utilizes two datasets: a 1D synthetic dataset for polynomial regression and a 2D dataset for Gaussian basis regression.

Dataset 1: 1D Univariate Data

This synthetic dataset is generated from a sinusoidal function with added Gaussian noise, simulating real-world data with inherent noise. The dataset contains a series of observations of a single variable ***x***, paired with target values ***t*** generated by applying the function ***t = sin(2πx) + noise***. The objective is to fit a polynomial function to this data, exploring the effects of polynomial degree on model flexibility and generalization. The data is divided into a 70% training and 30% testing split to evaluate model performance on unseen data.

Dataset 2: 2D Bivariate Data

This dataset consists of bivariate data observations, divided into training (70%) and testing (30%) sets. The 2D data will be used to implement Gaussian basis regression, with centers of the Gaussian basis functions determined by K-means clustering on the training data. The dataset allows us to evaluate model complexity by adjusting the number of basis functions and applying regularization to control overfitting. Gaussian basis functions are expected to capture underlying patterns in the data by concentrating the model's focus around dense data regions, identified through clustering.

In the following sections, we delve into the methodology and implementation details for each dataset, presenting results through graphical representations and analyzing model performance metrics across different levels of complexity and regularization.

3. Methodology

In this assignment, we use **Polynomial Regression** for 1-dimensional data and **Gaussian Basis Linear Regression** for 2-dimensional data. The primary objective is to explore the effects of model complexity and regularization on regression performance, particularly how these factors influence generalization. Below is a step-by-step breakdown of the methodology, covering data preprocessing, model training, parameter selection via cross-validation, and performance analysis.

3.1. Polynomial Regression (for 1D Data)

3.1.1 Model Overview

Polynomial regression is a form of linear regression where the model fits a polynomial function of degree M to the data. By varying the degree M, we can increase or decrease the model's flexibility:

- **Low-degree polynomials** produce simpler models that may underfit the data.
- **High-degree polynomials** allow for more flexibility but can lead to overfitting.

The model is expressed as:

$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{j=0}^{M} w_j x^j$$

where M represents the polynomial degree, and w_j are the weights or coefficients to be learned from the training data.

3.1.2 Regularization to Prevent Overfitting

To prevent overfitting in high-degree polynomials, we apply **regularization** (ridge regression). Regularization penalizes large weight values, effectively smoothing the model and reducing overfitting.

The regularized objective function is:

$$E_{\text{reg}}(\mathbf{w}) = \frac{1}{2N} \sum_{n=1}^{N} (y(x_n, \mathbf{w}) - t_n)^2 + \frac{\lambda}{2} \sum_{j=0}^{M} w_j^2$$

where λ is the regularization parameter. When λ = 0, the model behaves like standard polynomial regression without regularization. As λ increases, the model becomes more constrained, which can help control overfitting.

3.1.3 Cross-Validation for Model Complexity and Regularization

We use cross-validation to select the optimal:

- **Polynomial degree** M: Values of M range from 2 to 9 to study the effect of model complexity.
- **Regularization parameter** λ: Cross-validation helps determine the best λ value to balance fit quality and generalization.

3.1.4 Training and Optimization

1. **Objective Function**: The model minimizes the **mean squared error (MSE)**, defined as:

$$E(\mathbf{w}) = \frac{1}{2N}\sum_{n=1}^{N}(y(x_n, \mathbf{w}) - t_n)^2$$

When regularization is applied, this is modified to $E_{\text{reg}}(\mathbf{w})$ as shown above.

2. **Weight Estimation**: The model weights ($\mathbf{w}$) are computed using closed-form solutions available for both polynomial regression and regularized polynomial regression.

3.1.5 Evaluation Metrics

The model's performance is evaluated using:

- **Mean Squared Error (MSE)** on both training and test datasets.
- **Root Mean Squared Error (RMSE)** to allow easy comparison across different model configurations.

3.2. Gaussian Basis Linear Regression (for 2D Bivariate Data)

3.2.1 Model Overview

Gaussian Basis Linear Regression is a type of linear regression that transforms the input data using Gaussian basis functions centered around key points in the data. This technique allows for capturing non-linear relationships by transforming the input space.

The model is expressed as:

$$y(\mathbf{x}, \mathbf{w}) = \sum_{j=1}^{M} w_j \phi_j(\mathbf{x})$$

where $\phi_j(\mathbf{x}) = \exp\left(-\frac{|\mathbf{x}-\mu_\mathbf{j}|^2}{2\sigma^2}\right)$ represents the Gaussian basis function centered at $\mu_\mathbf{j}$, and σ is a spread parameter. M is the number of basis functions, which is a key complexity parameter in the model.

3.2.2 Determining Basis Centers with K-means Clustering

To position the Gaussian centers $\mu_\mathbf{j}$ effectively:

1. **K-means Clustering**: We use K-means clustering on the training data to determine M cluster centers. These cluster centroids serve as the centers of our Gaussian basis functions.
2. **Model Complexity (Number of Basis Functions)**: We vary M to study the impact of model complexity. Tested values include M = 2,4,8,16,32,128 and 256.

3.2.3 Regularization to Improve Generalization

Like polynomial regression, Gaussian basis regression can overfit when the number of basis functions M is high. We apply **L2 regularization** to the model weights to improve generalization. The regularized objective function is:

$$E_{\text{reg}}(\mathbf{w}) = \frac{1}{2N}\sum_{n=1}^{N}(y(\mathbf{x_n}, \mathbf{w}) - t_n)^2 + \frac{\lambda}{2}|\mathbf{w}|^2$$

where λ controls the regularization strength. Cross-validation is used to determine the best λ value for each model configuration.

3.2.4 Training and Optimization

1. **Objective Function**: The model minimizes the mean squared error $E(\mathbf{w})$, modified with regularization as shown above.
2. **Weight Estimation**: The weights are computed using closed-form solutions, leveraging the linear structure of the Gaussian basis expansion.

3.2.5 Evaluation Metrics

We evaluate the model using:

- **MSE and RMSE** on both training and test datasets to assess generalization and fit quality.
- **Scatter Plots of Model Output vs Target Output** for both datasets to visualize how well the model captures the data patterns.
- **Decision Region and Contour Plots** to visualize the Gaussian basis model's decision boundaries, which show how well the model captures regions of high density in the data.

3.3 Cross-Validation for Hyperparameter Tuning

Cross-validation is employed to select:

1. **Model Complexity**:
    - **Degree M** for polynomial regression.
    - **Number of basis functions M** for Gaussian basis regression.
2. **Regularization Parameter λ**: Cross-validation is used to find the optimal regularization strength for each configuration, balancing the trade-off between model fit and generalization.

3.4 Graphical Analysis and Performance Plots

We include several plots to assess model behavior:

1. **MSE and RMSE vs. Model Complexity**: To analyze the effect of increasing model flexibility on training and test error.
2. **Model Output vs Target Output Scatter Plots**: To compare the predicted vs actual values, visualizing the fit quality.
3. **Decision Region and Contour Plots** (for Gaussian basis model): These plots allow for visualization of how well the model captures different regions in the input space.

These analyses provide insights into how model complexity and regularization affect the performance and generalization of the models. The methodology emphasizes closed-form solutions and cross-validation to balance the complexity and regularization for optimal performance on unseen data.

4. Implementation for Dataset 1 – Univariate Data

Visualizing data using Scatter Plot

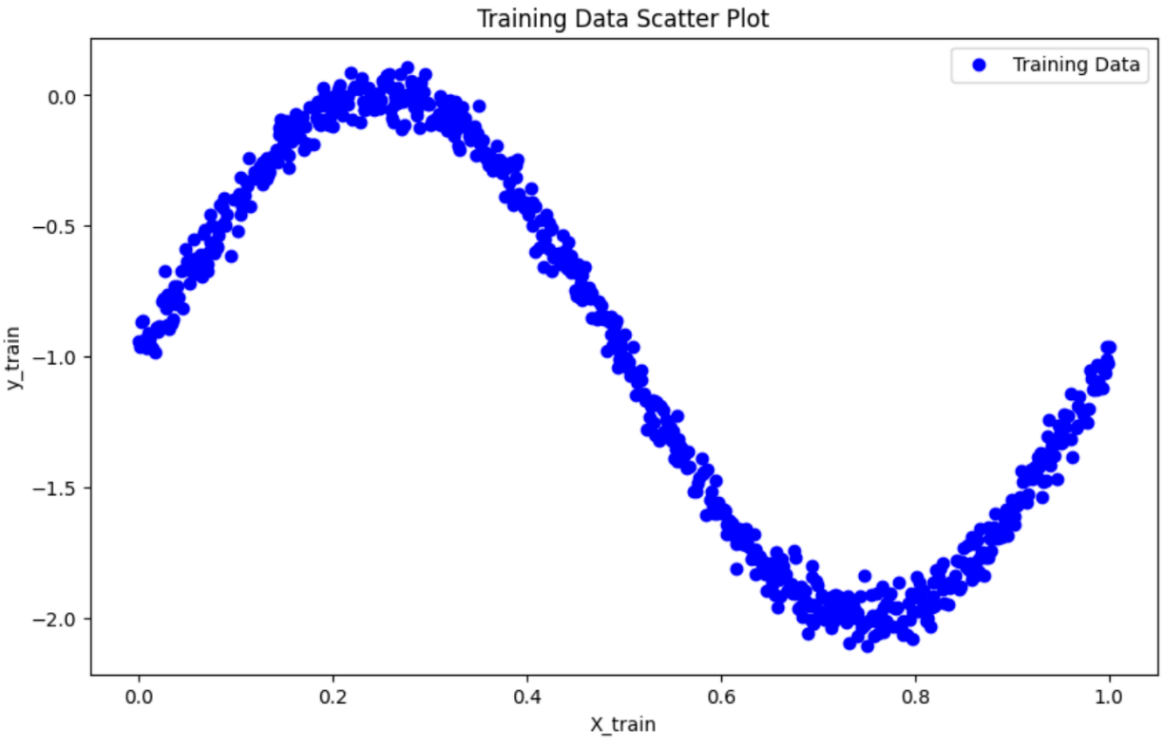


Figure 1 Scatter Plot on Training Data

Observations from Figure 1

- The scatter plot displays the relationship between the independent variable (***X_train***) and the dependent variable (***y_train***) in the training dataset.

- This initial visualization gives a clear view of the data distribution and any apparent relationship between (X) and (y). For polynomial regression, we're interested in seeing if the data might follow a curved pattern rather than a straight line, which would suggest a non-linear relationship.

Implementing Polynomial Curve Fitting

Creating Training Subsets:

- We define multiple subset sizes stored in a list called subset_sizes, which includes values [10, 50, 100, and the full training set length].
- For each size in subset_sizes, we create a subset of the training data by selecting the first size rows from the full training set (X_train_full and y_train_full). This step allows us to observe the impact of training subset size on the model's ability to fit polynomial curves.
- These subsets are used to investigate how model performance and overfitting behavior change with different amounts of data.

Looping Through Polynomial Degrees:

- For each training subset, we vary the polynomial degree from 2 to 9 to study the effect of model complexity.
- We create polynomial features using PolynomialFeatures(degree=degree), which transforms the independent variable X into polynomial terms up to the specified degree. For example, a degree-2 polynomial will include an X^2 term, a degree-3 polynomial will add both X^2 and X^3 terms, and so on.

Fitting the Model:

- After transforming X values to polynomial features, we use LinearRegression to fit the model on the transformed data (X_train_poly) and the subset target values (y_train_subset).
- The model is trained to minimize the error for the specified polynomial degree, creating a polynomial fit for each degree.

Plotting the Polynomial Fit:

- To visualize the fit, we generate a range of X values (X_range) covering the entire span of the training data.
- We transform X_range with polynomial features and use the model to predict the corresponding y-values (y_pred). This step allows us to plot a smooth curve representing each polynomial fit.
- Each plot includes:
    - **Scatter Points** for the original training subset, showing actual data points.

- **Polynomial Curves** for each degree (from 2 to 9), with labels for easy comparison.

- **Plot Insertion**: Each subset size has a dedicated plot showing polynomial fits for degrees 2 through 9. The plot title specifies the subset size, and a legend labels each polynomial degree.

Final Visualization:

- For each subset size, a plot visualizes polynomial fits across degrees 2 through 9, allowing a comparison of model complexity's impact on fit quality.

- Titles indicate the subset size, and legends label polynomial degrees for clear distinction.

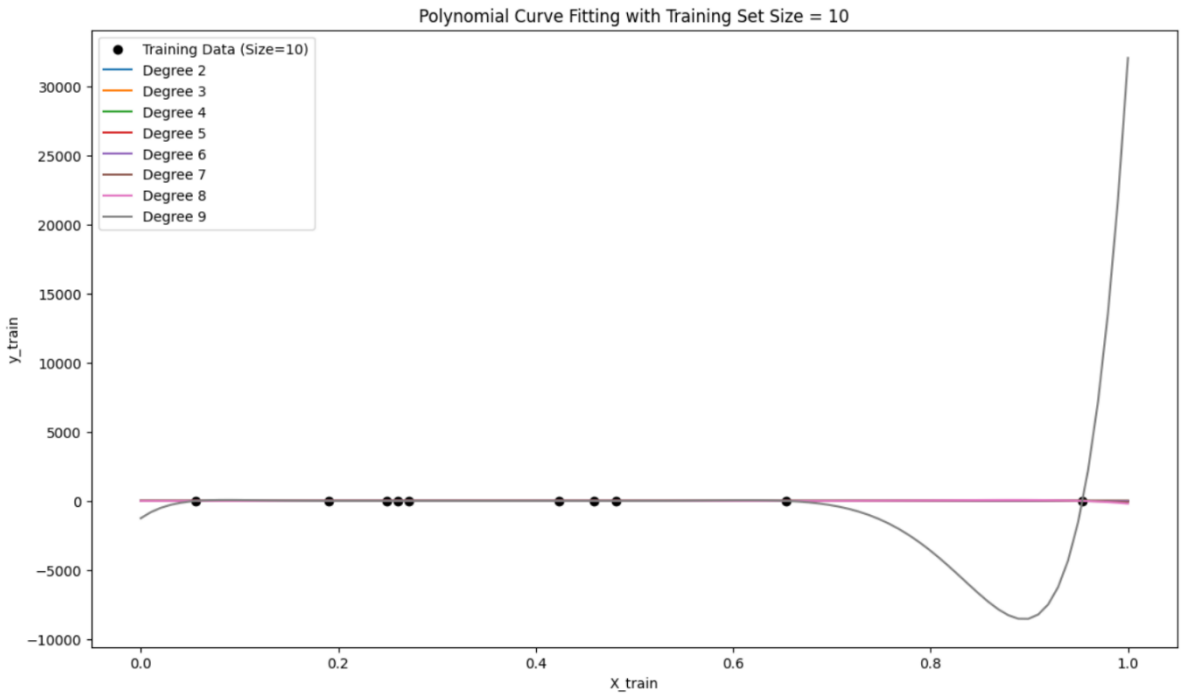


Figure 2 Polynomial Curve Fitting with Training Set Size = 10

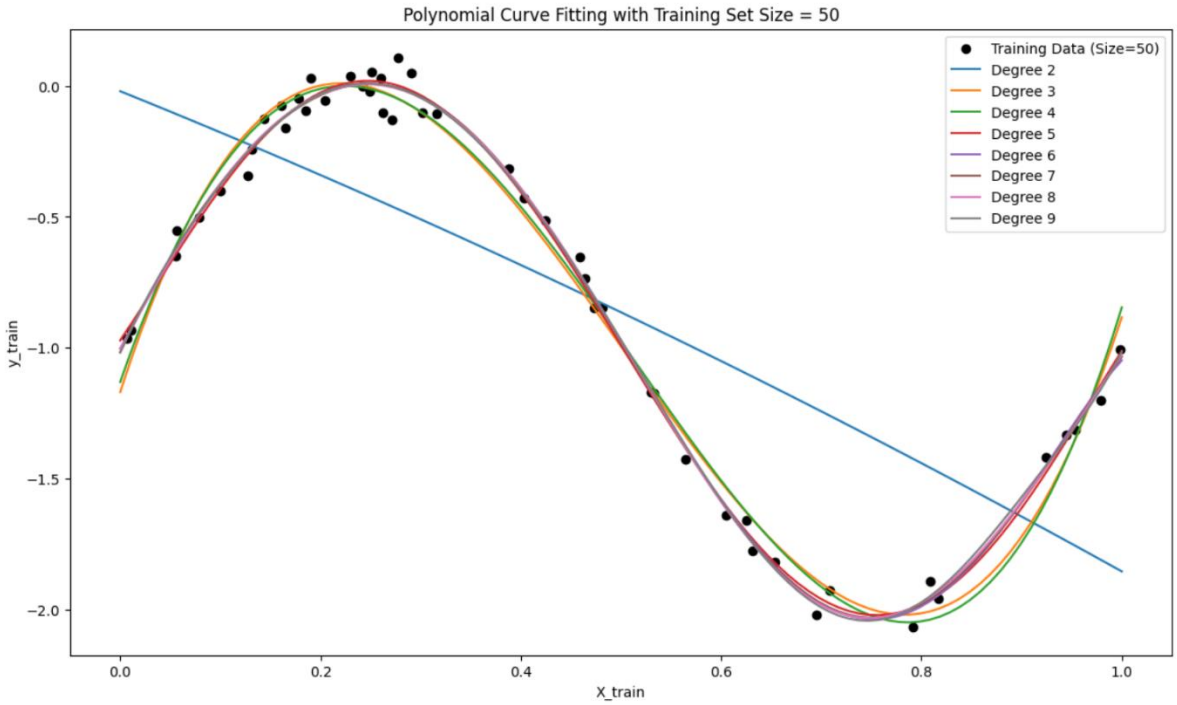


Figure 4 Polynomial Curve Fitting with Training Set Size = 50

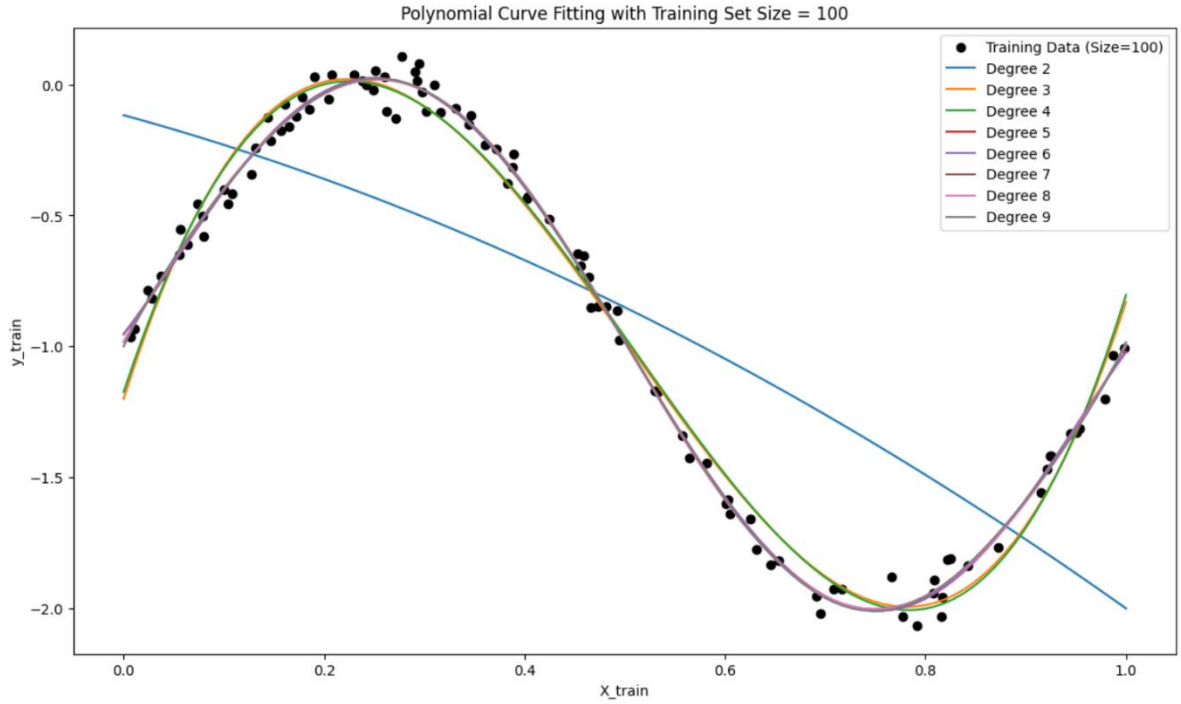


Figure 3 Polynomial Curve Fitting with Training Set Size = 100

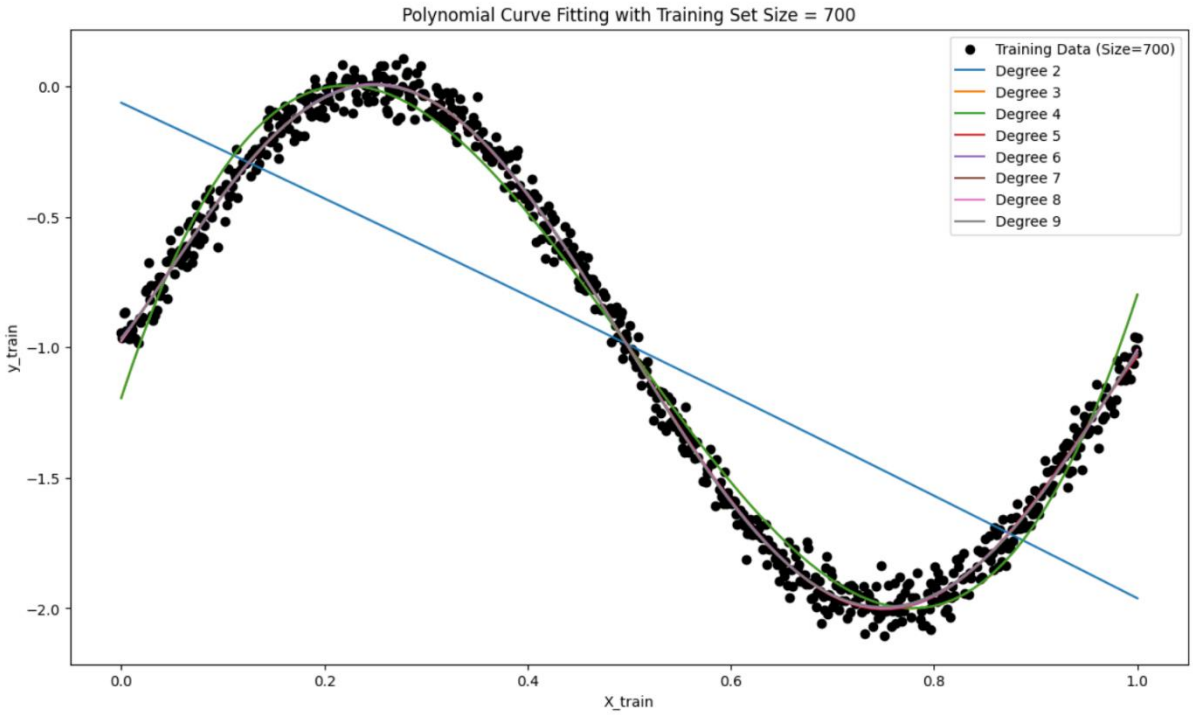


Figure 5 Polynomial Curve Fitting with Training Set Size = 700

Observations

1. **Effect of Training Data Size**:
    - **Small Subsets (10 and 50)**: With fewer data points, higher-degree polynomials often overfit, as they attempt to fit each point exactly. This overfitting leads to wavy, overly complex curves that do not generalize well.
    - **Larger Subsets (100 and Full Set)**: As the subset size increases, the models begin to capture the general trend more accurately, particularly with lower-degree polynomials. Overfitting becomes less pronounced for higher degrees, as the model has more data to learn the underlying trend.
2. **Impact of Polynomial Degree**:
    - **Low Degree (2 to 3)**: Lower-degree polynomials generally produce smoother, more generalized fits, capturing the overall trend of the data. They are effective across all subset sizes, especially when the data has a simpler underlying structure.

- **Medium Degree (4 to 6)**: These degrees capture moderate data complexity, balancing trend capture without excessive oscillation. They are effective in representing the underlying pattern without overfitting.
- **High Degree (7 to 9)**: Higher-degree polynomials often capture small variations and oscillations, which may not represent the true trend. Overfitting is particularly evident in smaller subsets, where the polynomial curves oscillate to match each data point.

Inferences

1. **Model Complexity and Overfitting**:
   - Higher-degree polynomials introduce more flexibility, allowing the model to fit the data closely, but they are prone to overfitting, especially on smaller subsets. Selecting an appropriate polynomial degree depends on the dataset size and complexity.
   - **Cross-validation** is essential to select the best degree that balances bias (underfitting) and variance (overfitting), allowing the model to generalize well to unseen data.
2. **Importance of Training Data Size**:
   - When data points are limited, higher-degree polynomials tend to overfit, while lower-degree models produce smoother, more stable fits. As the subset size grows, higher degrees start to fit the true pattern better, highlighting the importance of sufficient data to avoid overfitting.
   - **Regularization** (e.g., Ridge regression) could help control overfitting for higher degrees, especially on smaller subsets or noisier data.
3. **Bias-Variance Trade-Off**:
   - This implementation illustrates the bias-variance trade-off in polynomial regression, where low-degree models may underfit, and high-degree models can overfit. The balance between complexity and generalization depends on the subset size and underlying data structure, emphasizing the need for model complexity tuning and data sufficiency.

Implementing Regularization with Ridge Regression

To further improve the polynomial regression model's generalization, we implement Ridge regression, a regularization technique that penalizes large coefficient values, helping control model complexity, especially for high-degree polynomials.

Implementation

1. **Applying Ridge Regularization**:
    - For each polynomial degree from 2 to 9, we apply Ridge regression with different regularization strengths, denoted by alpha values (0.001, 0.01, 0.1, 1, and 10).
    - **Alpha Values**: Smaller alpha values indicate less regularization, allowing the model more flexibility, while larger values impose stronger regularization, reducing coefficient magnitudes and, consequently, the model's complexity.
2. **Fitting Polynomial Models**:
    - We transform the training data (X_train_full) into polynomial features for each degree using PolynomialFeatures. This transformation enables fitting polynomial models of increasing complexity, capturing non-linear relationships as the polynomial degree grows.
3. **Recording Model Coefficients**:
    - After fitting each model, we store the coefficients for each alpha value. Ridge regression penalizes larger coefficients, so examining how these coefficients vary with different alpha values and polynomial degrees helps illustrate regularization's impact on model flexibility.
4. **Plotting the Fitted Curves**:
    - For each polynomial degree, we create plots showing the fitted curves for different alpha values alongside the training data points. This allows a visual comparison of how the polynomial fits become smoother and less flexible as regularization strength increases.
    - **Plot Insertion**: Insert plots of the fitted curves for each degree, showcasing how different alpha values affect the model's complexity on the same training data.

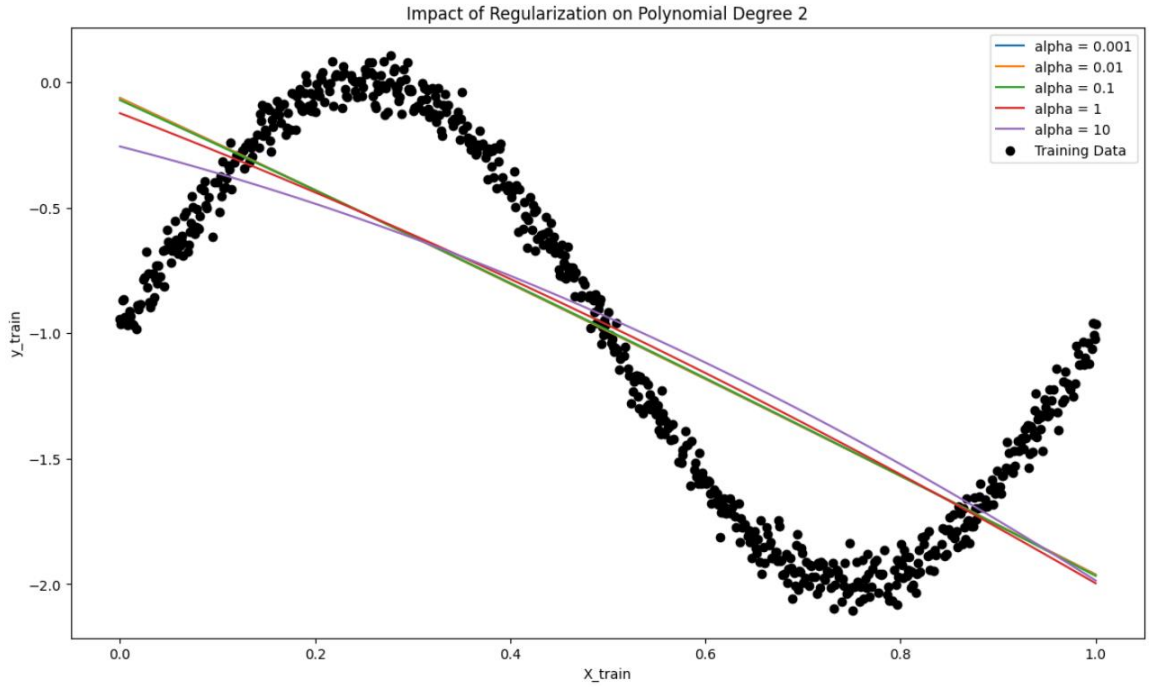


Figure 6 Impact of Regularization on Polynomial Degree 2

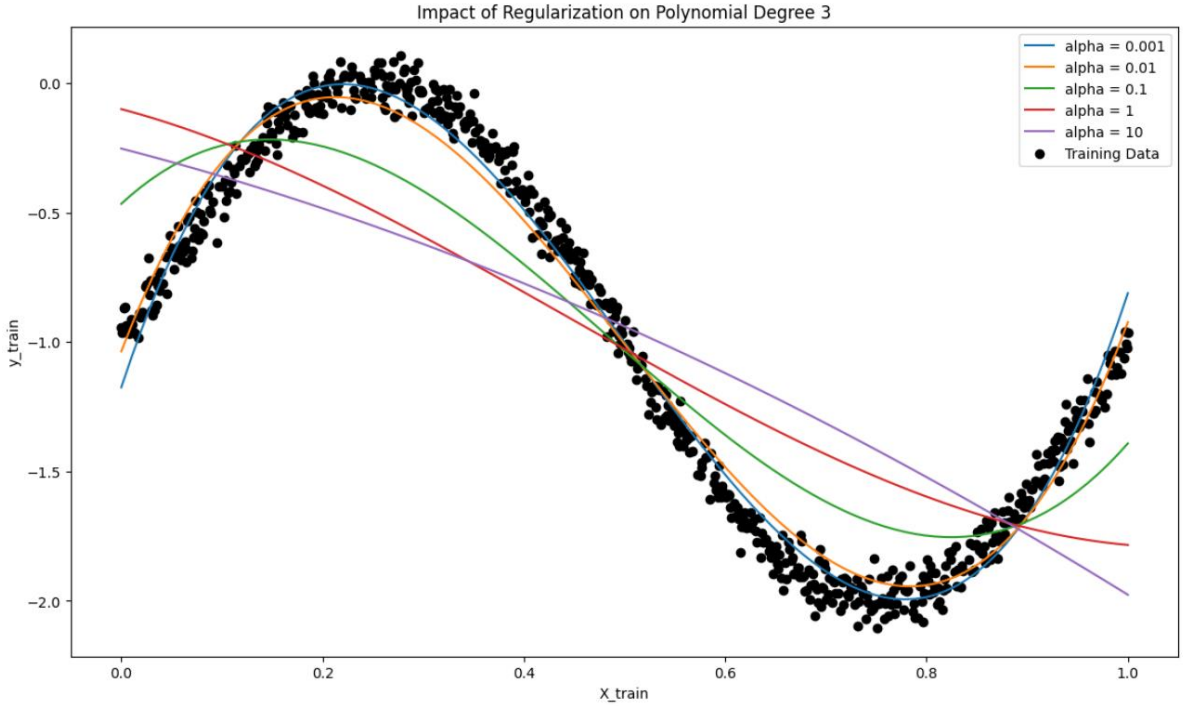


Figure 7 Impact of Regularization on Polynomial Degree 3

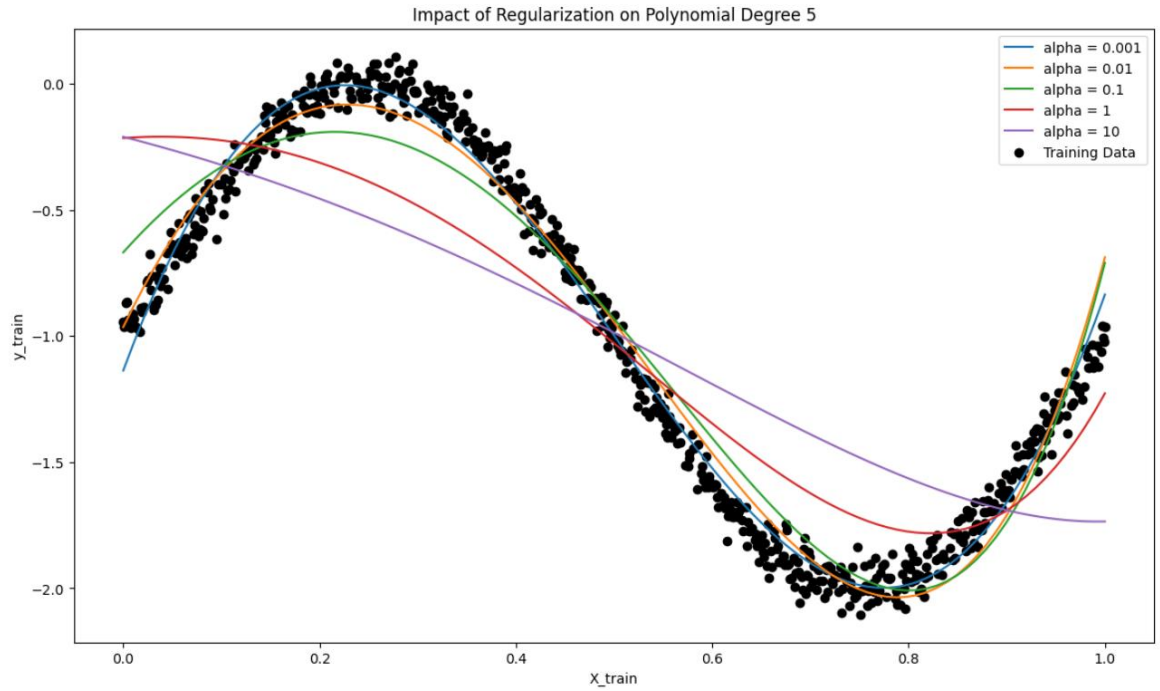


Figure 9 Impact of Regularization on Polynomial Degree 5

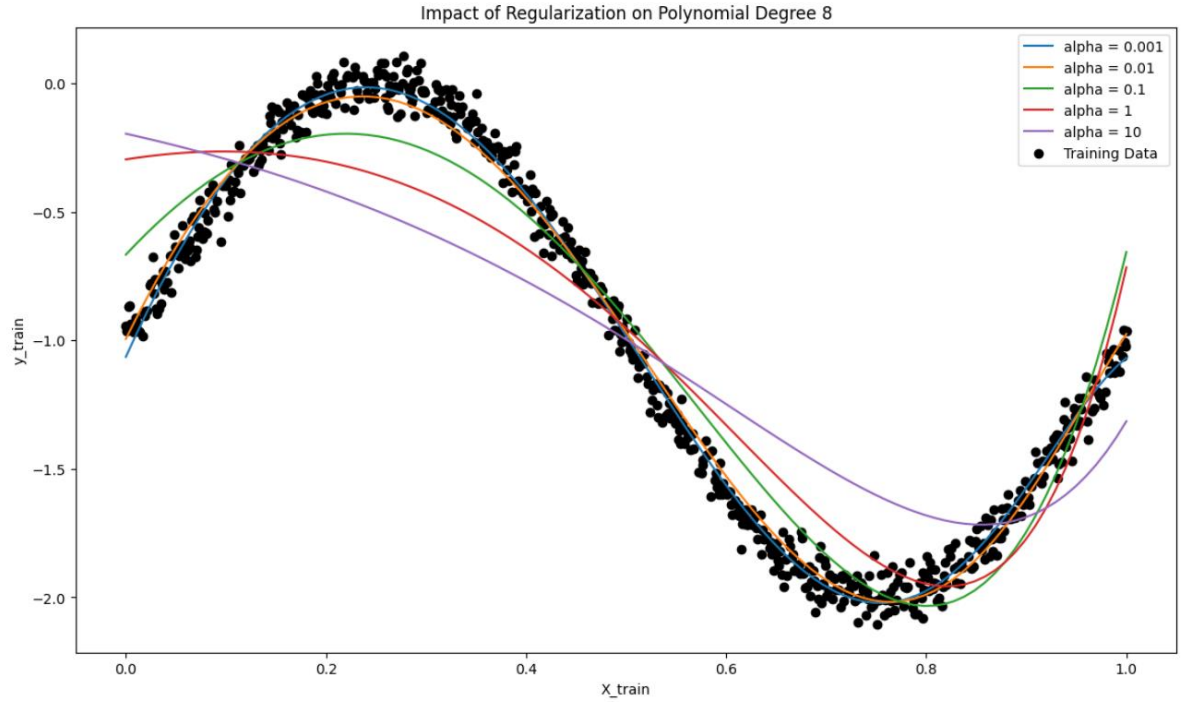


Figure 8 Impact of Regularization on Polynomial Degree 8

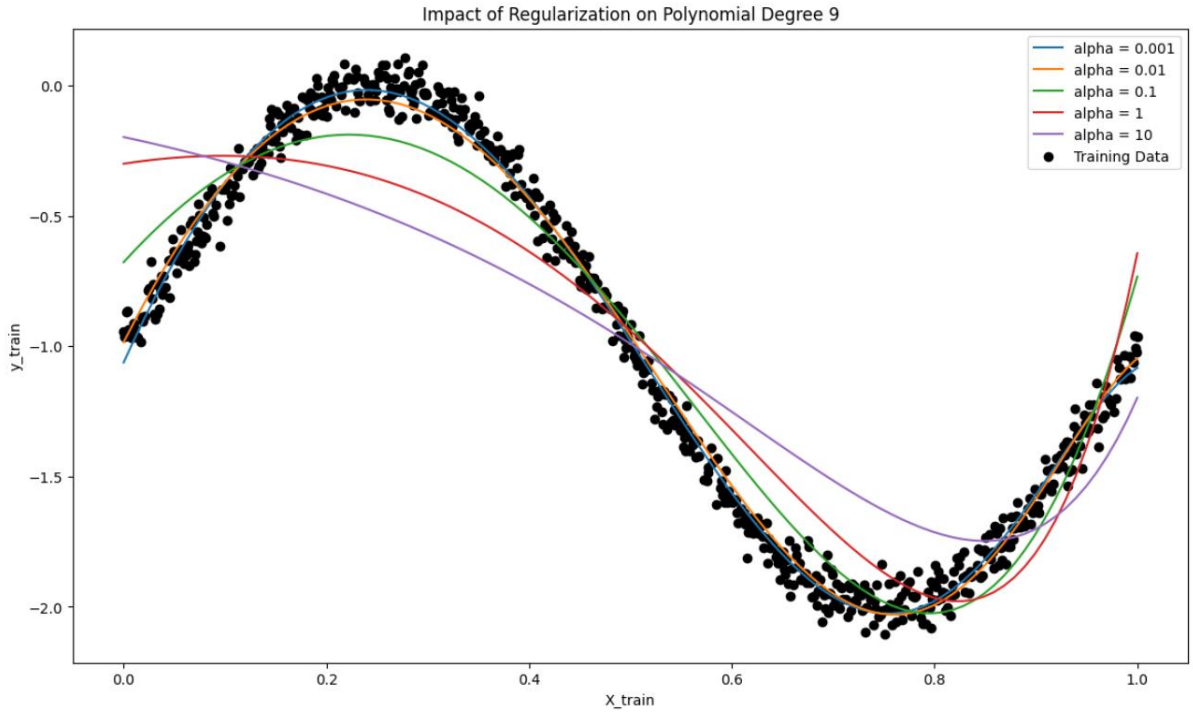


Figure 10 Impact of Regularization on Polynomial Degree 9

Observations

1. **Effect of Increasing Alpha on Model Complexity**:
    - As alpha increases, the polynomial fits become smoother and less flexible, with high-degree polynomials appearing more linear. Regularization reduces the influence of higher-order terms, making the model less prone to overfitting, particularly at higher polynomial degrees.
2. **Coefficient Shrinkage with Higher Alpha**:
    - The coefficient plots indicate that as alpha grows, the magnitude of polynomial coefficients, especially for higher-order terms, decreases. This shrinkage limits the influence of complex terms in the polynomial, helping prevent overfitting by simplifying the model.
3. **Impact of Polynomial Degree**:
    - For lower-degree polynomials (e.g., degree 2 or 3), the effect of regularization is relatively mild because these models are simpler and less likely to overfit. As the polynomial degree increases, regularization has a stronger effect, and higher alpha values lead to smoother curves with reduced oscillations, effectively reducing overfitting.

Inferences

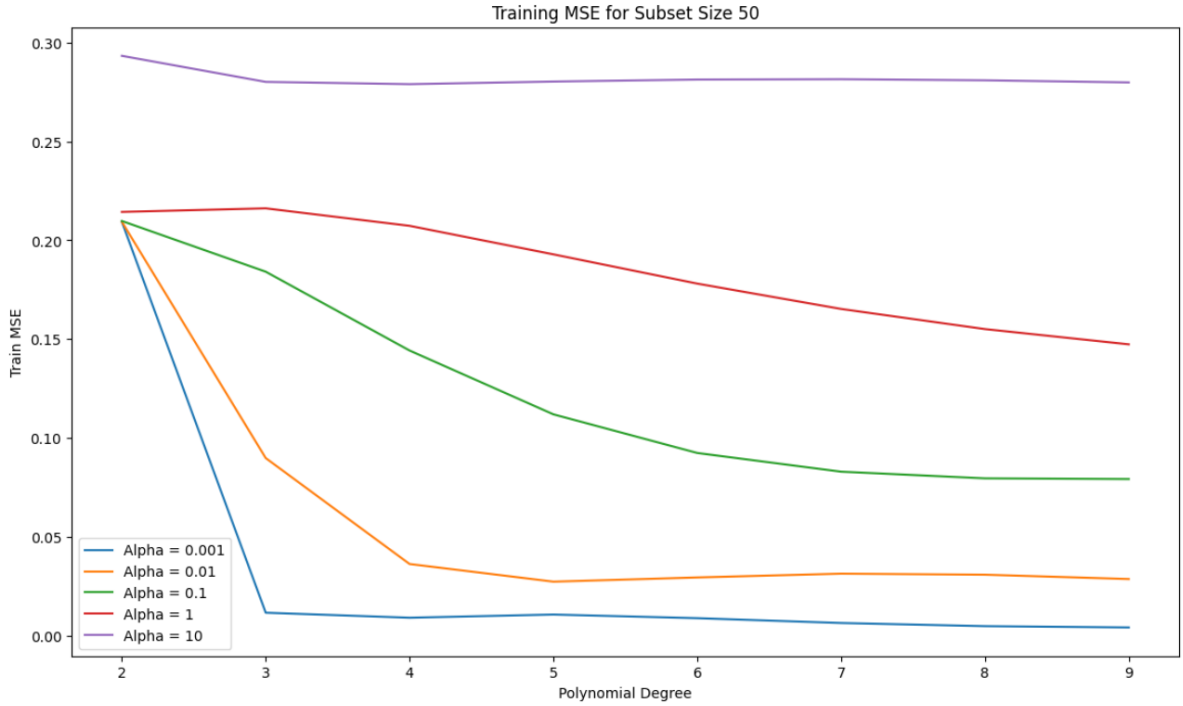
1. **Bias-Variance Trade-Off**:
    - Regularization helps balance the **bias-variance trade-off** in polynomial regression. Lower alpha values allow the model to fit complex patterns but may lead to high variance and overfitting. Higher alpha values, while reducing variance, increase bias as the model becomes overly simplistic and less adaptable to the data's nuances.
2. **Optimal Alpha Selection**:
    - The ideal alpha value strikes a balance between model flexibility and generalization. **Cross-validation** can help identify this optimal alpha, ensuring the model fits underlying patterns in the data without overfitting to noise.
3. **Importance of Regularization for High-Degree Polynomials**:

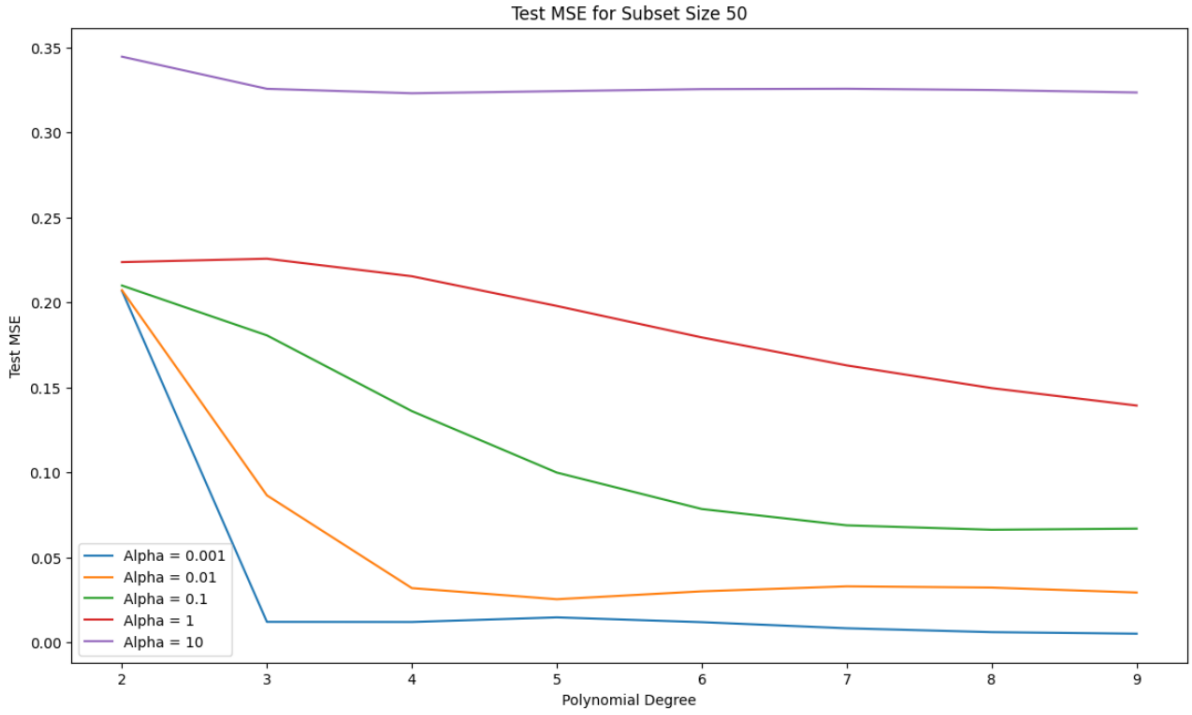
- Regularization is especially valuable for higher-degree polynomials, which are more prone to overfitting. By penalizing large coefficients, Ridge regression focuses the model on capturing significant data patterns, reducing the risk of overfitting.

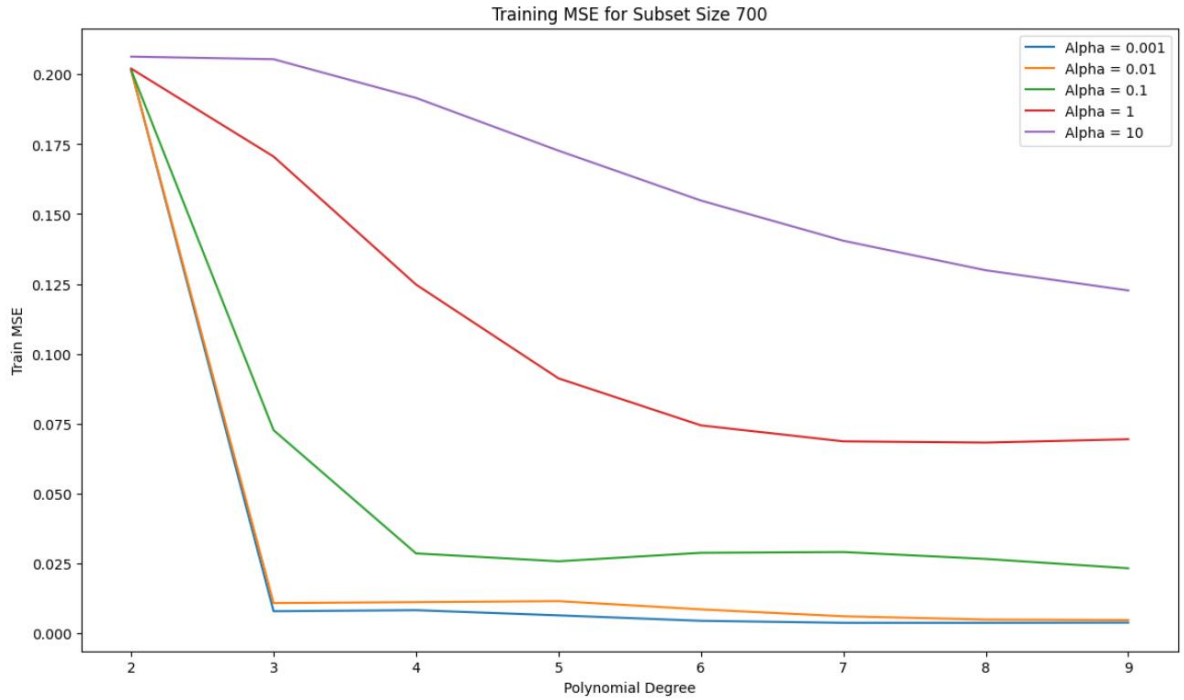
Implementing cross-validation Mean Squared Error

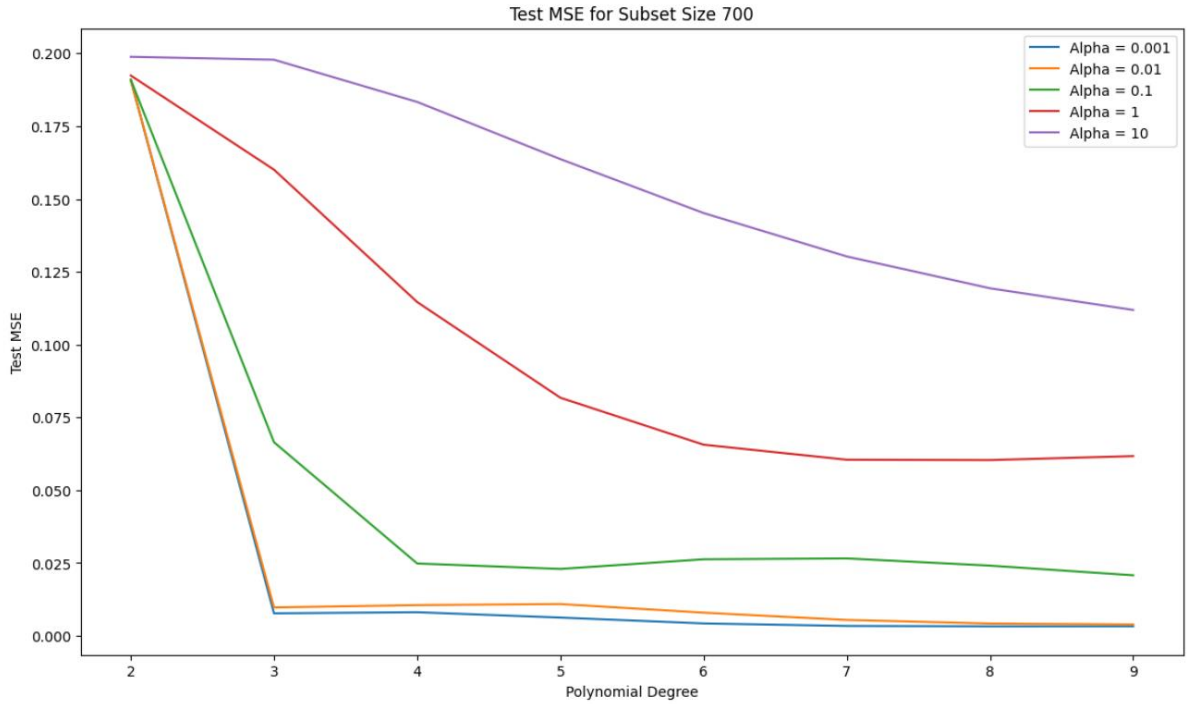
1. **Cross-Validation MSE Calculation**:
    - For each combination of training subset size, polynomial degree, and regularization parameter, we use **5-fold cross-validation** with cross_val_score to calculate the **cross-validation Mean Squared Error (CV MSE)**.
    - Cross-validation evaluates each model's generalization ability by splitting the subset into training and validation folds. This approach provides insight into which polynomial degree and alpha value yield the lowest validation error, indicating the best balance between complexity and generalization.
2. **Training MSE Calculation**:
    - Alongside CV MSE, we calculate the **training MSE (Train MSE)** for each model configuration. This metric shows the model's accuracy on the training subset, helping us compare training performance against cross-validation performance.
    - Discrepancies between Train MSE and CV MSE reveal important patterns: if Train MSE is significantly lower than CV MSE, the model is likely overfitting, while high errors on both indicate underfitting.
3. **Storing and Organizing Results**:
    - Results for each model configuration are stored in two lists, mse_train_results and mse_cv_results, which are later converted to DataFrames (mse_train_df and mse_cv_df) for convenient access and visualization.
4. **Plotting Cross-Validation MSE**:
    - For each subset size, a plot of **CV MSE vs. polynomial degree** is created, with separate lines for each alpha value. This visual helps illustrate how model complexity (polynomial degree) and regularization (alpha) affect validation error, aiding in the selection of the optimal degree and alpha that minimize CV MSE.
5. **Plotting Training MSE**:
    - Similarly, for each subset size, we plot **training MSE vs. polynomial degree** with lines for each alpha value. These plots show how training MSE responds to

increased polynomial degrees and various alpha values, highlighting any signs of overfitting or underfitting.






Training MSE for Subset Size 700
Alpha = 0.001
Alpha = 0.01
Alpha = 0.1
Alpha = 1
Alpha = 10
Train MSE
0.200
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
2
3
4
5
6
7
8
9
Polynomial Degree


Test MSE for Subset Size 700
Alpha = 0.001
Alpha = 0.01
Alpha = 0.1
Alpha = 1
Alpha = 10
Test MSE
0.200
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
2
3
4
5
6
7
8
9
Polynomial Degree

Observations

1. **Cross-Validation MSE Trends**:
    - **Low Alpha Values**: With low alpha values, higher-degree polynomials often yield lower CV MSE as they fit the data flexibly. However, CV MSE may increase for very high degrees, indicating overfitting where the model becomes overly complex and begins capturing noise in the data.
    - **High Alpha Values**: For larger alpha values, CV MSE generally stabilizes or increases as polynomial degree grows. This shows that regularization reduces overfitting by limiting the model's flexibility, particularly at higher degrees.
2. **Training MSE Trends**:
    - **Low Alpha Values and High Degrees**: At low alpha values, training MSE decreases substantially with increasing polynomial degree as the model captures more data variance. However, when training MSE is much lower than CV MSE, it suggests overfitting.
    - **High Alpha Values and Lower Degrees**: With higher alpha values, training MSE increases, indicating a reduction in model flexibility. For high degrees, training MSE tends to flatten with large alpha values, showing regularization's role in reducing the impact of higher-degree terms.
3. **Effect of Subset Size**:
    - **Small Subsets**: For smaller subsets (e.g., size 10), CV MSE remains high across polynomial degrees, highlighting the difficulty in generalizing with limited data. Regularization has a more pronounced effect, as high-degree polynomials tend to overfit small datasets without adequate regularization.
    - **Larger Subsets**: As subset size increases (50, 100, full set), CV MSE generally decreases, and the optimal polynomial degrees stabilize. With larger datasets, regularization is less critical, as the data quantity itself aids in preventing overfitting.

Inferences

1. **Optimal Polynomial Degree and Regularization**:
    - The best model configuration minimizes CV MSE while keeping Train MSE close to CV MSE, indicating a balanced model that generalizes well. Typically,

intermediate polynomial degrees (e.g., 3 to 6) and moderate alpha values achieve the best generalization across subset sizes.

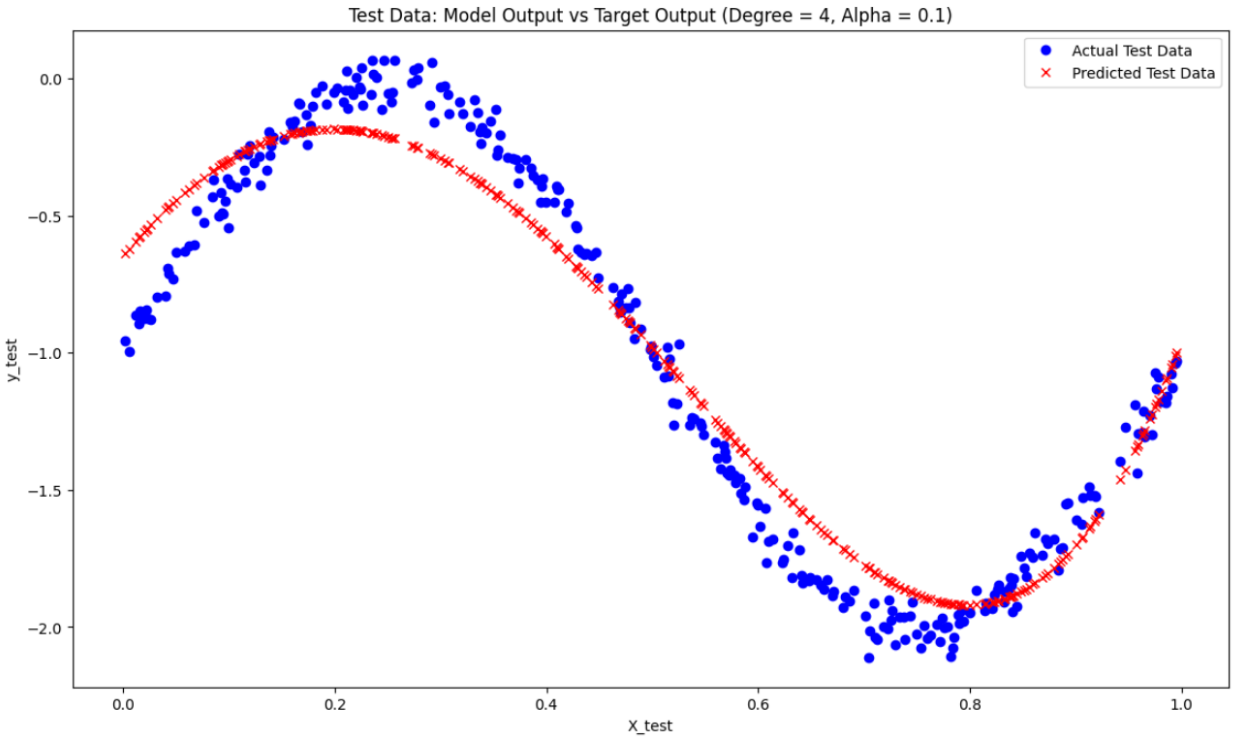
2. **Importance of Cross-Validation**:
    - Cross-validation is crucial for identifying the ideal model complexity and regularization, especially on small subsets prone to overfitting. By testing different combinations of polynomial degrees and alpha values, cross-validation helps select models that generalize better without relying solely on training data performance.
3. **Regularization's Impact on High-Degree Polynomials**:
    - Regularization is especially valuable for higher-degree polynomials where overfitting is likely. By reducing the influence of high-order terms, larger alpha values improve generalization by shrinking coefficients, thus focusing the model on significant patterns and reducing noise sensitivity.

Final Model Evaluation with Polynomial Degree and Regularization

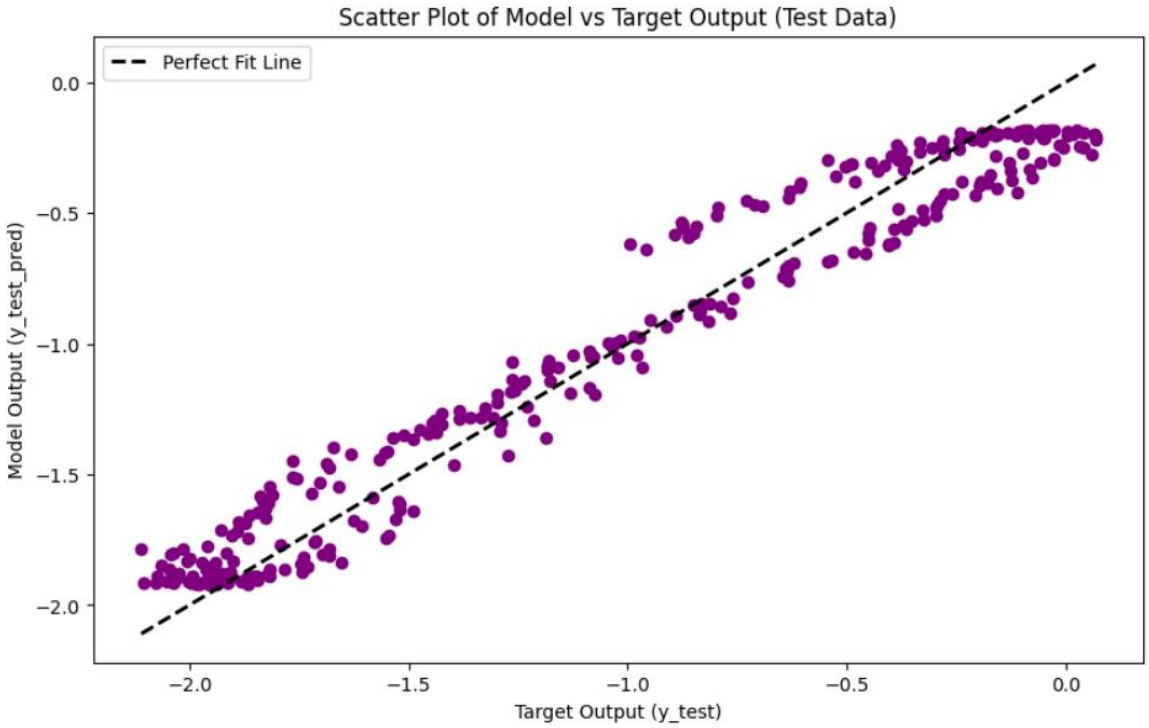
This section details the final model evaluation process using the optimal polynomial degree and regularization parameter (alpha) obtained from cross-validation. The goal is to validate the model's performance on both training and test data, ensuring that it generalizes well to new, unseen data.

1. **Transform Data Using Best Polynomial Degree**:
    - Using the optimal polynomial degree identified through cross-validation, we transform both the training and test datasets into polynomial features. This transformation enables the model to capture non-linear relationships in the data based on the best-fitting polynomial complexity.
2. **Fit the Best Model**:
    - The Ridge regression model is trained on the transformed training data with the selected regularization parameter (alpha). This alpha value balances model flexibility and regularization, preventing overfitting while retaining sufficient complexity to capture significant patterns in the data.

3. **Generate Predictions for Training and Test Data**:
    - After fitting the model, predictions are generated for both training (y_train_pred) and test datasets (y_test_pred). Comparing these predictions with actual target values allows us to assess how well the model generalizes.
4. **Plot Model Output vs Target Output**:
    - We create a plot to visually compare actual target values with the model's predicted values for both training and test data. **Red markers** represent predicted values, while **blue markers** represent actual target values.
    - **Plot Insertion**: This plot should display predicted values aligning closely with actual values if the model generalizes well. Close alignment of red and blue markers indicates that the model captures the underlying trend accurately.



5. **Scatter Plot for Target vs Model Output**:
    - **Scatter plots** are created for both training and test datasets, with the target values on the x-axis and model predictions on the y-axis.
    - A **45-degree line** is added as a reference for a perfect fit, where predictions exactly match the target values. Points lying near this line indicate accurate predictions, while points deviating from the line suggest potential overfitting (high accuracy on training but lower on test) or underfitting.
    - **Plot Insertion**: Scatter plots for both training and test data against the perfect fit line should be included to assess prediction accuracy and generalization.



Observations

1. **Model Output vs Target Output**:
    - **Training Data**: If the red markers (predicted values) are closely aligned with blue markers (actual values) on the training plot, it suggests that the model has learned the data pattern well without overfitting. The degree of alignment indicates how accurately the model captures the training data.
    - **Test Data**: Alignment of predicted and actual values in the test data indicates the model's generalization ability. If there are significant deviations between predicted and actual values, this may suggest overfitting or underfitting, where the model fails to capture the pattern in unseen data.
2. **Scatter Plot Analysis**:
    - **Training Data Scatter Plot**: Points near the 45-degree line indicate good prediction accuracy for the training set. If points deviate widely from this line, it may indicate potential underfitting or overfitting, depending on how the test data performs.
    - **Test Data Scatter Plot**: This plot shows the model's performance on unseen data. If points are near the 45-degree line, it suggests strong generalization. However, if test data points deviate significantly from this line while training points are close, this suggests overfitting. Widespread deviations on both plots indicate underfitting.

Inferences

1. **Model Generalization and Complexity**:
    - If points in both training and test scatter plots are tightly clustered around the 45-degree line, this implies a good balance between complexity and generalization. This alignment suggests that the selected polynomial degree and regularization parameter effectively capture the data's structure without overfitting.
2. **Evaluating Model Accuracy**:
    - Close alignment between actual and predicted values across both training and test datasets implies that the model generalizes well. However, if only the training data points are close to the line and test data points deviate, this indicates overfitting. Conversely, if both sets of points are far from the line, the model may be underfitting.

3. **Validating Model Selection**:
    - The scatter plots and model output plots provide visual confirmation that the selected model parameters are appropriate. If both training and test plots show good alignment with actual data, this suggests that the chosen polynomial degree and alpha value achieve optimal performance across both seen and unseen data.

Conclusion

Through this final evaluation using the selected polynomial degree and regularization parameter, we confirm the model's effectiveness in capturing the data's patterns without overfitting. Comparing the model's predictions to actual values on both training and test data, as well as examining scatter plots against the perfect fit line, provides a comprehensive view of model performance. These visualizations validate that the model complexity and regularization are appropriately tuned for accurate and reliable predictions.

6. Conclusions

The experiments conducted across multiple datasets reveal important insights

7. References

- **Bishop, C. M. (2006).** *Pattern Recognition and Machine Learning.* Springer.
- **Duda, R. O., Hart, P. E., & Stork, D. G. (2000).** *Pattern Classification.* Wiley.
- **Murphy, K. P. (2012).** *Machine Learning: A Probabilistic Perspective.* MIT Press.
- **Mitchell, T. M. (1997).** *Machine Learning.* McGraw-Hill.